



EXAM **HEIST**



CompTIA

# EXAM **HEIST**

## Premium exam material

Get certification quickly with the Exam Heist Premium exam material.  
Everything you need to prepare, learn & pass your certification exam easily. Lifetime free updates

First attempt guaranteed success.

<https://www.ExamHeist.com>

# Microsoft

(GH-300)

GitHub Copilot

Total: **188 Questions**

Link: <https://examheist.com/papers/microsoft/gh-300>

**Question: 1****Exam Heist**

Which Microsoft ethical AI principle is aimed at ensuring AI systems treat all people equally?

- A. Privacy and Security
- B. Fairness
- C. Reliability and Safety
- D. Inclusiveness

**Answer: B****Explanation:**

The correct answer is **B. Fairness**.

Fairness, within the context of Microsoft's ethical AI principles, focuses directly on ensuring that AI systems do not discriminate or exhibit bias against any individual or group of people. This principle is critical for promoting equitable outcomes and preventing AI from perpetuating existing societal inequalities. It addresses potential biases embedded in training data, algorithms, and deployment strategies. An AI system designed with fairness in mind will strive to treat everyone equally, irrespective of their race, gender, religion, or other protected characteristics.

Privacy and Security (A) is concerned with protecting personal data and ensuring the safety of AI systems from malicious actors, but not primarily with equal treatment. Reliability and Safety (C) emphasizes the dependable and secure functioning of AI, focusing on minimizing risks and unintended consequences. Inclusiveness (D), while related, focuses on designing AI that is accessible and benefits a broad range of people, including those with disabilities or who are otherwise marginalized. While Inclusiveness contributes to fairness, it doesn't explicitly tackle the core principle of treating everyone equally. The core tenet of the question specifically relates to "treating all people equally," aligning perfectly with the "Fairness" principle. Fairness is about actively mitigating biases and ensuring just outcomes, making it the most pertinent answer among the choices provided.

For further research on Microsoft's Ethical AI principles, refer to the following resources:

**Microsoft AI Principles:** <https://www.microsoft.com/en-us/ai/responsible-ai>

**Question: 2****Exam Heist**

What can be done during AI development to minimize bias?

- A. Collect massive amounts of data for training.
- B. Focus on accuracy of the data.
- C. Use diverse data, fairness metrics, and human oversight.
- D. Improve on the computational efficiency and speed.

**Answer: C****Explanation:**

**C. Use diverse data, fairness metrics, and human oversight.**

## Strategies for Minimizing AI Bias

Minimizing bias requires a multi-faceted strategy that addresses the issue at every stage of the AI development lifecycle. Option C covers the three most critical components:

### 1. Diverse Data (Input/Training Stage)

**Problem:** AI models learn patterns from the data they are trained on. If the training data disproportionately represents certain demographics or excludes others (e.g., lack of data for women, specific racial groups, or non-English speakers), the resulting model will inherit and amplify those societal biases.

**Solution:** Use diverse, representative, and balanced datasets that accurately reflect the population or scenarios the AI will interact with. This is the foundational step to prevent bias.

### 2. Fairness Metrics (Model Development/Testing Stage)

**Problem:** Traditional AI optimization focuses solely on metrics like "accuracy" or "error rate" across the entire dataset. A highly "accurate" model can still be highly biased if its errors are concentrated among a specific subgroup (e.g., better loan application approval prediction for one gender over another).

**Solution:** Use fairness metrics (like disparate impact, equal opportunity difference, or demographic parity) to measure the model's performance specifically across sensitive subgroups. This forces developers to treat fairness as a technical requirement alongside accuracy.

### 3. Human Oversight (Evaluation/Deployment Stage)

**Problem:** Bias can be subtle and difficult to detect through automated metrics alone, especially when it concerns subjective real-world outcomes.

**Solution:** Incorporate human oversight and review, including domain experts, ethicists, and representatives from the communities potentially affected by the AI. This ensures that the system is evaluated for real-world harm and ethical implications before and after deployment.

## Why Other Options are Insufficient

**A. Collect massive amounts of data for training:** While large datasets are necessary for high performance, a massive amount of biased data only results in a massive amount of bias. Size does not guarantee diversity or fairness.

**B. Focus on accuracy of the data:** "Accuracy of the data" is vague. If it means data quality, that's important, but data can be high-quality (accurate labels) and still be heavily biased (unrepresentative samples). The focus needs to be on representativeness and diversity, not just quality.

**D. Improve on the computational efficiency and speed:** This relates solely to the performance and resource utilization of the model, having no direct bearing on its ethical fairness or bias.

## Question: 3

**Exam Heist**

Why is it important to ensure the security of the code used in Generative AI (Gen AI) tools?

- A. Ensuring code security prevents unauthorized access and potential data breaches.
- B. Ensuring code security enables the AI system to handle larger datasets effectively.
- C. Ensuring code security maintains the integrity of the AI system.
- D. Ensuring code security supports the development of more advanced AI features.

**Answer: A**

**Explanation:**

A. Ensuring code security prevents unauthorized access and potential data breaches.

Security of the code used in Generative AI tools is critical because:

Gen AI tools may process sensitive source code, secrets, or proprietary logic.

If security is weak, this information could be leaked, exposed, or accessed by unauthorized parties.

Protecting code ensures confidentiality and reduces risk of breaches.

**The Official Exam Answer:**

For the specific question, "**Why is it important to ensure the security of the code used in Generative AI (Gen AI) tools?**", the correct answer according to the GitHub certification key is **A. Ensuring code security prevents unauthorized access and potential data breaches.**

Why Option A is correct in the exam context:

While **Integrity (Option C)** is a core theoretical pillar of AI security, the GH-300 exam focuses heavily on **Responsible AI** and **Data Privacy** for enterprise customers. In the context of "Code used in Gen AI tools," the exam designers prioritize the **Confidentiality** aspect.

1. **Sensitive Data Exposure:** Gen AI tools often have access to proprietary source code, internal APIs, and environment variables. If that "code-in-use" is insecure, it directly leads to unauthorized access to the company's intellectual property.
2. **Enterprise Risk:** For a business, the primary "harm" of an insecure AI tool is not just a logic error (Integrity), but the **legal and financial liability** of a data breach where proprietary code or secrets are leaked to the model provider or external actors.
3. **Microsoft/GitHub Framing:** Under Microsoft's "Privacy and Security" pillar for Responsible AI, the emphasis is placed on ensuring that the system is "secure from unauthorized access" to protect user data and privacy.

**Question: 4**

**Exam Heist**

A social media manager wants to use AI to filter content. How can they promote transparency in the platform's AI operations?

- A. By providing clear explanations about the types of content the AI is designed to filter and how it arrives at its conclusion.
- B. By relying on a well-regarded AI development company.
- C. By regularly updating the AI filtering algorithm.
- D. By focusing on user satisfaction with the content filtering

**Answer: A**

**Explanation:**

The correct answer is A because transparency in AI operations, especially for content filtering, requires clear communication with users about how the system works. Providing explanations about the types of content the AI targets and the reasoning behind its filtering decisions builds trust and allows users to understand and

potentially appeal decisions if they disagree. This transparency helps mitigate concerns about bias, censorship, and errors in the AI's judgment. Options B, C, and D, while potentially contributing to a better-performing AI, don't directly address the issue of transparency. Relying on a well-regarded company (B) doesn't guarantee transparency. Regular updates (C) improve accuracy but don't necessarily inform users about the process. Focusing on user satisfaction (D) measures outcomes but doesn't explain how those outcomes are achieved. Transparency in AI is crucial for ethical AI development and deployment. Without it, users are left in the dark about how their content is being treated and filtered, potentially leading to mistrust and resentment. In social media context, if users understand filtering mechanism they can modify their content accordingly, thus helping them communicate their ideas more efficiently.

Here are some authoritative links for further research on responsible AI and transparency:

Why B is incorrect: Trusting a reputable company is good practice, but it does not make the day-to-day operations or decision-making process visible to the end-users. Why C is incorrect: Updating the algorithm falls under Reliability and Safety or continuous improvement, but updates alone do not help users understand how the decisions are actually being made. Why D is incorrect: User satisfaction is a business performance metric; a system can be satisfying to use while remaining completely secretive about how its logic functions.

**Microsoft's Responsible AI Principles:** <https://www.microsoft.com/en-us/ai/responsible-ai>

**Google AI Principles:** <https://ai.google/principles/>

**OECD AI Principles:** <https://www.oecd.org/going-digital/ai/principles/>

## Question: 5

Exam Heist

How does GitHub Copilot Chat utilize its training data and external sources to generate responses when answering coding questions?

- A. It primarily relies on the model's training data to generate responses.
- B. It combines its training data set, code in user repositories, and external sources like Bing to generate responses.
- C. It uses user-provided documentation and exclusively to generate responses.
- D. It primarily uses search results from Bing to generate responses.

**Answer: B**

**Explanation:**

**Why option B is the correct answer**

**Training data foundation** – The underlying language model has been pre-trained on a massive corpus of publicly available code, documentation, and technical literature. This gives it broad coding knowledge and syntax awareness.

**Repository-aware context** – When a user interacts with CopilotChat, the service streams the contents of the current workspace (open files, symbols, recent edits). This contextual information is injected into the prompt so the model can tailor suggestions to the specific project, language, and coding patterns in use.

**External data enrichment** – For up-to-date or niche information, CopilotChat can invoke a live search component (powered by Bing) that retrieves recent documentation, library release notes, or community discussions. The retrieved snippets are then merged with the model's internal knowledge before generating a response.

**Dynamic synthesis** – The final output is produced by blending the three sources: (1) the model's pre-trained world knowledge, (2) the user's repository context, and (3) any freshly fetched external material. This hybrid approach yields answers that are both generic-competent and project-specific, while staying current.

### Why the other options are less suitable

**OptionA** – Relying only on the training set ignores the crucial repository context and the ability to fetch fresh information, leading to generic or outdated suggestions that may not align with the user’s codebase.

**OptionC** – The claim that CopilotChat uses exclusively user-provided documentation is inaccurate; user-provided files are just one of several context sources, and the system also draws on the model’s own knowledge and external search results.

**OptionD** – While Bing-driven search is part of the pipeline, it is not the primary driver of responses. The model still relies heavily on its trained parameters and the live code context; search is used selectively to augment, not replace, the core generation process.

### References

GitHub Copilot Chat overview and architecture: <https://docs.github.com/en/copilot/copilot-chat>

How Copilot uses Bing search to augment responses: <https://learn.microsoft.com/en-us/bing/search-apis/overview>

These sources detail the integration of training data, repository context, and external search in Copilot Chat’s response generation.

### Question: 6

Exam Heist

Which principle emphasizes that AI systems should be understandable and provide clear information on how they work?

- A. Fairness
- B. Accountability
- C. Transparency
- D. Inclusiveness

### Answer: C

#### Explanation:

#### Justification

**Transparency** directly refers to the requirement that AI systems be understandable, with clear documentation and explanations of their operation, data sources, and decision logic.

**Option C (Transparency)** precisely matches this definition and is the principle most often cited in AI ethics frameworks (e.g., EU AI Act, OECD AI Principles).

**Fairness (A)** focuses on avoiding bias and ensuring equitable outcomes, which is unrelated to explainability.

**Accountability (B)** concerns responsibility for outcomes and the ability to attribute decisions to individuals or organizations, not the clarity of internal workings.

**Inclusiveness (D)** emphasizes broad access and representation for diverse user groups, again distinct from the need for clear internal explanations.

Therefore, option **C** is the only principle that explicitly emphasizes understandability and clear informational disclosure about system functioning.

### References

1. European Commission, Proposal for a Regulation on Artificial Intelligence (EU AI Act), 2021.

### Question: 7

Exam Heist

Which of the following is a risk associated with using AI?

- A. AI replaces the need for developer opportunities in most fields.
- B. AI eliminated the need for data privacy regulations.
- C. AI algorithms are incapable of perpetuating existing biases.
- D. AI systems can sometimes make decisions that are difficult to interpret.

**Answer: C**

**Explanation:**

**Justification**

**OptionC is the best choice** because it highlights a critical misconception that can lead to unmanaged bias in AI deployments. When stakeholders assume that AI algorithms are incapable of perpetuating existing biases, they may forgo bias-assessment practices, allowing hidden discriminatory outcomes to persist unchecked. This complacency is a well-documented risk in responsible-AI frameworks, as it can cause inequitable results and legal exposure.

**OptionA** overstates AI's impact on employment. While AI automates certain tasks, it also creates new roles and augments existing ones; the claim that it "replaces the need for developer opportunities in most fields" is an exaggeration and not a primary risk associated with AI use.

**OptionB** misrepresents regulatory obligations. Data-privacy regulations (e.g., GDPR, CCPA) remain applicable regardless of AI adoption; AI does not eliminate the need for such compliance, so this is not a valid risk description.

**OptionD** describes a genuine technical limitation (lack of interpretability), but the question asks for a risk associated with using AI, and the most salient risk emphasized in responsible-AI guidance is the potential for unchecked bias due to false assumptions of neutrality. Hence, OptionC aligns best with the intended answer.

**References**

Microsoft Responsible AI resources: <https://learn.microsoft.com/en-us/azure/machine-learning/responsible-ai-overview>

European Commission guidance on AI bias and risk mitigation: [https://ec.europa.eu/info/strategy/priorities-2019-2024/digital-transformation/ai-committee\\_en](https://ec.europa.eu/info/strategy/priorities-2019-2024/digital-transformation/ai-committee_en)

### Question: 8

Exam Heist

What types of prompts or code snippets might be flagged by the GitHub Copilot toxicity filter? Each correct answer presents part of the solution. (Choose two.)

- A. Hate speech or discriminatory language (e.g., racial slurs, offensive stereotypes)
- B. Sexually suggestive or explicit content
- C. Code that contains logical errors or produces unexpected results
- D. Code comments containing strong opinions or criticisms



**Answer: AB**

**Explanation:**

#### **Technical Justification**

##### **OptionA – Hate speech or discriminatory language**

The Copilot toxicity filter scans user-provided prompts for language that promotes hate, discrimination, or harassment (e.g., racial slurs, ethnic stereotypes). Such content violates Microsoft's responsible AI policies and is explicitly blocked or flagged to prevent the generation of harmful outputs.

##### **OptionB – Sexually suggestive or explicit content**

Prompts containing sexual innuendo, pornographic descriptors, or explicit language are also filtered out. The model is designed to refuse or safe-complete any request that could produce sexually explicit material, aligning with content-safety standards.

##### **OptionC – Code that contains logical errors or produces unexpected results**

Logical errors or unexpected behavior are not a toxicity concern; they are addressed by standard debugging, testing, or code-review practices. The toxicity filter does not target syntactic or algorithmic correctness, only potentially harmful language.

##### **OptionD – Code comments containing strong opinions or criticisms**

Expressing personal opinions or criticisms in comments is permissible unless it includes hateful or harassing language. Strong opinions alone do not trigger the toxicity filter; only language that meets the criteria of A or B would be flagged.

#### **Conclusion**

The filter's purpose is to detect and block harmful language — specifically hate speech/discrimination (A) and sexually explicit content (B). Options C and D represent legitimate coding issues or stylistic choices that are handled outside the toxicity-filter scope.

#### **References**

GitHub Copilot Safety & Moderation Documentation: <https://learn.microsoft.com/en-us/github-copilot/safety>  
Responsible AI principles for Azure AI services: <https://learn.microsoft.com/en-us/azure/ai-services/content-moderation/overview>

#### **Question: 9**

**Exam Heist**

How can the concept of fairness be integrated into the process of operating an AI tool?

- A. Focusing on accessibility will ensure fairness.
- B. Training AI data and algorithms to be free from biases will ensure fairness.
- C. Regularly monitoring the AI tool's performance will ensure fairness in its outputs.
- D. Focusing on collecting large datasets for training will ensure fairness.

**Answer: A**

**Explanation:**

**Why option A is the most appropriate**

**Accessibility aligns with fairness** – Fairness in AI operation is often expressed as equitable access and usability for all user groups, regardless of ability, language, or context. By designing the tool to be accessible (e.g., supporting screen readers, multilingual interfaces, adjustable input methods), you ensure that the benefits of the AI are not limited to a subset of users, which directly embodies the principle of fairness.

**Compliance with inclusive design standards** – Accessibility requirements are codified in standards such as WCAG2.2 and Section508. Incorporating these from the operational phase makes the AI output usable by diverse populations, thereby mitigating systematic exclusion that would otherwise constitute unfair treatment.

**Holistic view of fairness** – Rather than focusing solely on data bias or performance metrics, accessibility addresses the end-to-end user experience, guaranteeing that all stakeholders can interact with and benefit from the AI system on equal footing.

### Why the other options are less suitable

**Option B** – While reducing bias in training data and algorithms is essential for ethical AI, it does not guarantee fairness in the operational context. An unbiased model can still be delivered through an inaccessible interface, thereby excluding certain users and producing unfair outcomes.

**Option C** – Continuous performance monitoring helps detect drift and degradation, yet it primarily serves to maintain accuracy and reliability. Fairness requires more than correct outputs; it demands that those outputs are available and meaningful to all user groups, which monitoring alone does not assure.

**Option D** – Larger datasets can improve model coverage, but size alone does not eliminate skewed representations or guarantee equitable access. Over-reliance on dataset size may propagate existing biases and does not address usability barriers that prevent certain users from receiving the AI's benefits.

### Conclusion

Integrating fairness into AI operation most directly involves ensuring the tool is accessible to all intended users, making option **A** the best answer. This approach embeds fairness into the very way the AI is experienced, rather than treating it as a secondary technical concern.

### References

Microsoft Responsible AI Dashboard – Accessibility considerations: <https://learn.microsoft.com/en-us/azure/machine-learning/responsible-ai-dashboard>

WCAG2.2 Standard (Web Content Accessibility Guidelines): <https://www.w3.org/TR/WCAG22/>

### Question: 10

Exam Heist

What are the potential risks associated with relying heavily on code generated from GitHub Copilot? Each correct answer presents part of the solution. (Choose two.)

- A. GitHub Copilot may introduce security vulnerabilities by suggesting code with known exploits.
- B. GitHub Copilot's suggestions may not always reflect best practices or the latest coding standards.
- C. GitHub Copilot may increase development lead time by providing irrelevant suggestions.
- D. GitHub Copilot may decrease developer velocity by requiring too much time in prompt engineering.

**Answer: AB**

**Explanation:**

**Correct options: A and B**

**A – Security vulnerabilities:** Copilot draws from publicly available repositories that may contain code with

known defects or exploits. When its completions are accepted without thorough review, these insecure patterns can be introduced into production code, increasing the attack surface of the application.

**B – Misalignment with best practices / latest standards:** The model is trained on a historical code base and therefore may suggest outdated APIs, anti-patterns, or coding conventions that have been superseded by newer language releases or industry standards. Accepting such suggestions uncritically can lead to maintainability issues and non-compliant code.

### Why C and D are less suitable

**C – Increase development lead time:** While irrelevant suggestions can cause extra iteration, Copilot's primary design goal is to accelerate coding by providing context-aware snippets. The occasional mismatch does not fundamentally extend project timelines; the risk is more about quality than schedule.

**D – Decrease velocity via prompt engineering:** Effective Copilot usage typically requires minimal prompt effort — developers simply accept or reject suggestions. The learning curve is low, and the tool is integrated into the IDE, so the overhead is generally negligible compared with the productivity gains.

### Key Takeaway

The primary concerns when relying on Copilot are security-related suggestions and deviation from up-to-date coding standards, both of which necessitate rigorous code review and validation.

### References

GitHub Copilot Security Overview: <https://docs.github.com/en/copilot/security>

Secure coding best practices for Copilot: <https://docs.github.com/en/copilot/security-best-practices>

## Question: 11

Exam Heist

Why might a Generative AI (Gen AI) tool create inaccurate outputs?

- A. The Gen AI tool is programmed with a focus on creativity over factual accuracy.
- B. The Gen AI tool is experiencing downtime and is not fully recovered.
- C. The training data might contain biases or inconsistencies.
- D. The Gen AI tool is being overloaded with too many requests at once.

**Answer: C**

**Explanation:**

**Justification**

**OptionC – Training data may contain biases or inconsistencies** – Generative AI models learn patterns from massive corpora of text that are curated from public sources. Those sources inevitably contain factual errors, contradictory statements, and social biases. When the model synthesizes new content, it can reproduce or amplify those imperfections, leading to inaccurate or misleading outputs. This is a well-documented source of hallucinations and factual errors in large language models.

**OptionA – Creativity over factual accuracy** – While some models are tuned to generate more diverse or imaginative responses, they are still fundamentally trained to predict likely sequences of text, not to ignore truth. Creativity does not systematically override factual grounding; errors arise from the data and model architecture rather than a deliberate design choice to sacrifice accuracy.

**OptionB – Downtime/recovery status** – Operational outages affect availability, not the substantive content of a response. Even when the service is fully operational, the model can still produce erroneous answers, making downtime an irrelevant cause of inaccuracy.

**OptionD – Overload with requests** – High request volume may increase latency or trigger throttling, but it does not inherently change the model’s internal probability distributions. Overload can degrade performance (e.g., longer response times), yet it does not directly cause factual mistakes; the underlying model behavior remains unchanged.

**Conclusion** – The root cause of inaccurate outputs is most directly tied to the quality and representativeness of the training data, which may embed biases and inconsistencies that propagate into generated content. Therefore, option **C** is the most appropriate answer.

## References

**Microsoft Documentation – Understanding Hallucinations in Generative AI:** <https://learn.microsoft.com/en-us/azure/ai-services/content-generation/concepts/hallucinations>

**Microsoft Responsible AI – Mitigating Bias and Inconsistencies:** <https://learn.microsoft.com/en-us/azure/ai-services/responsible-ai/concepts/bias-mitigation>

## Question: 12

Exam Heist

What should developers consider when relying on GitHub Copilot for generating code that involves statistical analysis?

- A. GitHub Copilot will automatically correct any statistical errors found in the user’s initial code.
- B. GitHub Copilot can independently verify the statistical significance of results.
- C. GitHub Copilot can design new statistical methods that have not been previously documented.
- D. GitHub Copilot’s suggestions are based on statistical trends and may not always apply accurately to specific datasets.

**Answer: D**

**Explanation:**

### Why option D is correct

GitHubCopilot generates suggestions by completing code patterns that it has seen in public repositories; it does **not** evaluate the mathematical correctness or domain-specific validity of those patterns. Consequently, its outputs may reflect statistical trends from the training data but can misapply or oversimplify statistical concepts for a given dataset. Developers must therefore review, validate, and possibly adjust any statistical code Copilot proposes.

**A** – Copilot does **not** automatically correct statistical errors; it lacks built-in error-checking or validation logic for statistical methods.

**B** – It cannot independently verify statistical significance; hypothesis testing and p-value calculations require domain knowledge and external libraries.

**C** – Copilot does **not** invent new statistical techniques; its outputs are limited to concatenations and variations of patterns present in the training corpus.

Therefore, the only statement that accurately captures Copilot’s role for statistical code is **D**: its suggestions are based on statistical trends and may not always apply accurately to specific datasets.

## References

GitHub Copilot documentation: <https://docs.github.com/en/copilot>

### Question: 13

Exam Heist

What is the primary role of the '/optimize' slash command in Visual Studio?

- A. Translates code into a more performant language.
- B. Automatically formats the code according to the selected style guide.
- C. Summarizes your documentation into more maintainable and readable formats.
- D. Enhances the performance of the selected code by analyzing its runtime complexity.

**Answer: D**

**Explanation:**

The correct answer is D: Enhances the performance of the selected code by analyzing its runtime complexity.

The /optimize slash command in Visual Studio, when used with GitHub Copilot, is designed to identify potential performance bottlenecks within a given code snippet. It goes beyond simple formatting or language translation. Instead, it leverages Copilot's AI capabilities to analyze the code's algorithmic complexity and suggest improvements that can lead to faster execution times and reduced resource consumption.

This analysis often involves identifying inefficient loops, redundant calculations, or suboptimal data structures. Copilot then provides specific recommendations, such as suggesting the use of more efficient algorithms, alternative data structures, or parallelization techniques, to optimize the code's performance. The command focuses directly on improving the code's runtime efficiency. While better code formatting and documentation can indirectly lead to maintainability and, therefore, indirectly better performance, the direct effect of /optimize is performance enhancement through runtime complexity analysis.

Cloud computing concepts support this explanation. Cloud environments often prioritize performance and resource utilization. Optimized code reduces the consumption of CPU cycles, memory, and network bandwidth in the cloud. A service like GitHub Copilot benefits cloud deployment by aiding developers in writing code that efficiently utilizes the cloud resources, lowering costs and improving application responsiveness. While direct documentation on /optimize for Visual Studio with GitHub Copilot might be sparse, searching for "GitHub Copilot performance optimization" will yield resources relating to AI-assisted performance tuning.

### Question: 14

Exam Heist

Which GitHub Copilot plan could an Azure DevOps organization use without requiring a GitHub Enterprise license?

- A. GitHub Copilot Enterprise
- B. GitHub Copilot for Azure DevOps
- C. Copilot Teams
- D. GitHub Copilot Individual

**Answer: B**

**Explanation:**

## B. GitHub Copilot for Azure DevOps.

GitHub Copilot for Azure DevOps is specifically designed for use inside Azure DevOps and does not require a GitHub Enterprise license.

All other Copilot plans (Enterprise, Teams, Individual) apply to GitHub, not Azure DevOps, and do not integrate natively with Azure DevOps pipelines or repos the same way.

### Question: 15

Exam Heist

Which of the following steps correctly demonstrates how to establish an organization-wide policy for GitHub Copilot Business to restrict its use to certain repositories?

- A. Create a copilot.policy file in each repository
- B. Create a copilot.policy in the .github repository
- C. Configure the policies in the organization settings
- D. Apply policies through the GitHub Actions configuration

**Answer: C**

#### Explanation:

The correct answer is **C. Configure the policies in the organization settings.**

GitHub Copilot Business policies are centrally managed at the organization level within GitHub. This allows administrators to define and enforce usage restrictions, including limiting Copilot's access to specific repositories, across the entire organization. This centralized approach simplifies management and ensures consistency in policy application. Options A, B, and D do not align with the documented mechanisms for managing GitHub Copilot Business policies organization-wide.

A copilot.policy file within each repository or in the .github repository would be ineffective because the GitHub Copilot Business policies are applied at the organizational level through the GitHub platform's settings. Utilizing GitHub Actions for applying these policies is also incorrect, as this isn't the designed method for Copilot Business control. GitHub Actions can automate other aspects of repository management, but not the central Copilot policies. Centralized control allows for easy updates and enforcement across all relevant repositories. The organization settings provide the control plane for managing these features. This aligns with the principles of centralized policy management common in cloud computing, where a single point of control governs access and usage across an entire infrastructure.

#### Further Research:

1. **GitHub Copilot Business Documentation:** <https://docs.github.com/en/copilot/overview-of-github-copilot/about-github-copilot-business>

### Question: 16

Exam Heist

What type of information can you retrieve through GitHub Copilot Business Subscriptions via REST API? Each correct answer presents part of the solution. (Choose two.)

- A. View code suggestions for a specific user

- B. List all GitHub Copilot seat assignments for an organization
- C. Get a summary of GitHub Copilot usage for organization members
- D. List of all unsubscribed GitHub Copilot members within an organization

**Answer: BC**

**Explanation:**

**The two correct answers are:**

- B. List all GitHub Copilot seat assignments for an organization
- C. Get a summary of GitHub Copilot usage for organization members

REST API Copilot Endpoints

The GitHub Copilot REST API is divided into two main categories: Copilot User Management and Copilot Metrics.

### **1. Copilot User Management (Seat Assignments)**

The REST API provides endpoints to manage who has access to a Copilot license (seat) within an organization.

List all GitHub Copilot seat assignments for an organization (B): This allows administrators to retrieve a list of all users who currently have an active Copilot license assigned to them.

Other Actions: The API also allows for adding and removing users or teams from the Copilot subscription.

### **2. Copilot Metrics (Usage Summary)**

The REST API includes endpoints for Copilot metrics, allowing administrators to monitor adoption and activity.

Get a summary of GitHub Copilot usage for organization members (C): This allows administrators to retrieve data for the last 100 days, including:

Numbers of active users and engaged users.

Breakdowns of usage by language and IDE.

This provides a high-level summary of how the Copilot investment is being utilized.

### **Why A and D are Incorrect**

A. View code suggestions for a specific user: The API provides metrics about usage (e.g., number of completions accepted), but it does not allow for retrieving the actual content of code suggestions given to a specific user. This is a privacy-sensitive area and is not exposed via the management API.

D. List of all unsubscribed GitHub Copilot members within an organization: The API focuses on the active seats and usage metrics. While you can infer who is not assigned a seat by comparing the seat assignments list (B) with the total organization member list, there is no single, dedicated API endpoint to list only "unsubscribed" members.

## **Question: 17**

**Exam Heist**

What is the best way to share feedback about GitHub Copilot Chat when using it on GitHub Mobile?



- A. The feedback section on the GitHub website.
- B. By tweeting at GitHub's official X (previously known as Twitter) account.
- C. Use the emojis in the Copilot Chat interface.
- D. The Settings menu in the GitHub Mobile app.

**Answer: C**

**Explanation:**

The correct answer is C: Use the emojis in the Copilot Chat interface. Here's why:

GitHub Copilot Chat, particularly within mobile applications like GitHub Mobile, is designed with integrated feedback mechanisms to allow users to directly communicate their experiences with the tool. Emojis within the chat interface serve as a quick and efficient way to express satisfaction or dissatisfaction with generated code suggestions or answers. This immediate feedback loop helps GitHub improve Copilot's performance and relevance. Using emojis directly within the chat interface links specific responses to your sentiment about them.

Options A, B, and D are less effective for providing targeted feedback specific to Copilot Chat interactions within GitHub Mobile. While the GitHub website might have a general feedback section (A), it wouldn't be contextually linked to your specific interaction with Copilot Chat. Tweeting at GitHub's official account (B) is public and lacks a structured format for feedback, making it difficult to track and address systematically. While the Settings menu in the GitHub Mobile app (D) may have feedback options, it likely offers a broader feedback mechanism unrelated to specific Copilot Chat responses. Therefore, emojis in the interface provide the most direct and context-aware method for sending your feedback directly to the Copilot Chat team.

This is a critical aspect of continuous improvement in cloud-based AI tools like Copilot. User feedback is essential for refining algorithms, models, and overall functionality. A built-in feedback mechanism in the user interface speeds up this process and makes it much more relevant. The context from the particular interaction helps the developers better understand what kind of feedback is being given.

To learn more about GitHub Copilot and its feedback mechanisms, refer to the official GitHub Copilot documentation: <https://github.com/features/copilot>. While specific documentation on emoji-based feedback within GitHub Mobile's Copilot Chat may be limited publicly, the principle of in-app feedback for iterative AI improvements is a widely adopted cloud computing best practice. Look for updates and announcements regarding Copilot Chat's features directly within the GitHub Mobile app or on GitHub's official blog.

**Question: 18**

**Exam Heist**

What specific function does the '/fix' slash command perform?

- A. Proposes changes for detected issues, suggesting corrections for syntax errors and programming mistakes.
- B. Converts pseudocode into executable code, optimizing for readability and maintainability.
- C. Generates new code snippets based on language syntax and best practices.
- D. Initiates a code review with static analysis tools for security and logic errors.

**Answer: A**

**Explanation:**

The '/fix' slash command in GitHub Copilot is specifically designed to address existing issues in code. Its



primary function is to propose changes that rectify identified problems. This includes suggesting corrections for common coding errors like syntax mistakes, incorrect variable assignments, or logical flaws. Think of it as a proactive debugging assistant embedded directly into your coding workflow. By automatically identifying and offering solutions to these issues, `/fix` helps developers write cleaner, more reliable code with less effort.

Options B, C, and D are incorrect because they describe functionalities handled by other Copilot features. While Copilot can generate new code (option C) or assist with tasks that touch on code review (option D), `/fix` narrows its focus to directly correcting existing code imperfections. Similarly, converting pseudocode (option B) isn't its core functionality. It focuses on refining already existing code, not transcribing it from a conceptual state.

Essentially, `/fix` acts as a real-time code doctor, diagnosing problems and prescribing solutions directly within your development environment. This allows for faster iteration, fewer debugging cycles, and ultimately, more productive coding sessions. This functionality aligns with the broader trend of AI-assisted development tools that aim to enhance developer efficiency and reduce error rates.

To delve deeper into GitHub Copilot slash commands and their specific functionalities, you can refer to the official GitHub Copilot documentation and related resources:

**GitHub Copilot Documentation:** <https://docs.github.com/en/copilot> (Specifically, look for information about slash commands and their purpose. While the documentation may not explicitly detail `/fix` in exhaustive detail, you'll find descriptions of the commands and their roles within the Copilot ecosystem. Look for explanations surrounding code correction and issue resolution.)

### Question: 19

Exam Heist

Which GitHub Copilot pricing plans include features that exclude your GitHub Copilot data like usage, prompts, and suggestions from default training GitHub Copilot? (Choose two.)

- A. GitHub Copilot Codespace
- B. GitHub Copilot Business
- C. GitHub Copilot Individual
- D. GitHub Copilot Enterprise

**Answer: BD**

**Explanation:**

The answer identifies the GitHub Copilot pricing plans that allow for excluding your organization's data from the default training process of GitHub Copilot's AI models. This is a crucial feature for enterprises concerned with data privacy, intellectual property protection, and compliance with data residency regulations.

Option B, GitHub Copilot Business, is designed for teams and organizations requiring enhanced control over their data. The documentation explicitly states that data generated through GitHub Copilot Business is not used to train the base models, ensuring that proprietary code remains confidential. This aligns with the cloud computing principle of data sovereignty and control.

Option D, GitHub Copilot Enterprise, takes this further, offering even more granular controls and data privacy options. Similar to Copilot Business, user code snippets and prompts are not used for public model training. This allows large organizations to leverage the benefits of AI-powered code completion while maintaining stringent data security and governance. Copilot Enterprise also includes advanced features for managing policies and access control.

Option A, GitHub Copilot Codespace, is primarily a cloud-based development environment. While integrated with GitHub Copilot, Codespace itself doesn't define data training policies for Copilot. It's the underlying Copilot subscription that governs the data usage.

Option C, GitHub Copilot Individual, is intended for personal use. Data from individual users is used to train and improve the overall GitHub Copilot model. Therefore, it doesn't meet the requirement of excluding data from default training. This highlights the different requirements and pricing for individual vs. enterprise usages.

Therefore, GitHub Copilot Business and Enterprise provide the data privacy features necessary for organizations to confidently adopt the tool without compromising sensitive information, distinguishing them from the Individual tier.

Authoritative Links:

GitHub Copilot Business: <https://github.com/features/copilot/business>

GitHub Copilot Enterprise: <https://github.com/features/copilot/enterprise>

### Question: 20

Exam Heist

When using an IDE with a supported GitHub Copilot plug-in, which Chat features can be accessed from within the IDE? Each correct answer presents part of the solution. (Choose two.)

- A. Explain code and suggest improvements
- B. Find out about releases and commits
- C. Generate unit tests
- D. Plan coding tasks

**Answer: AC**

**Explanation:**

**A — Explain code and suggest improvements.**

Copilot Chat can analyze the code in your editor and explain:

what it does

how it works

potential bugs

ways to refactor or optimize

**C — Generate unit tests**

Copilot Chat supports commands like:

/tests

/generate-tests

to create unit tests based on the code in your workspace.

### Why Other Options Are Incorrect

#### B. Find out about releases and commits

Why: Tracking software release tags, pull request statuses, or Git commit histories is a repository management capability. These are viewed directly on the GitHub.com platform interface or using specialized version control extensions, rather than via the inline Copilot code assistant chatbot.

#### D. Plan coding tasks

Why: High-level project planning, creating agile backlogs, or charting development sprint milestones are handled by project management frameworks (like GitHub Issues, Projects, or Jira). Copilot Chat assists with the execution of code blocks, not the enterprise planning of overall workflows.

### Question: 21

Exam Heist

Which Copilot Enterprise features are available in all commercially supported IDEs?

- A. Knowledge bases
- B. Chat
- C. Inline suggestions
- D. Pull request summaries

#### Answer: BC

#### Explanation:

B — Chat

Copilot Chat is fully supported across major IDEs, including:

Visual Studio

VS Code

JetBrains IDEs

Neovim

C — Inline suggestions

Inline code completions (the original Copilot feature) work in all supported IDEs.

### Question: 22

Exam Heist

What two options navigate to configure duplicate detection? Each correct answer presents part of the solution. (Choose two.)

- A. Organization settings → Copilot → Policies
- B. Enterprise settings → Copilot → Policies
- C. Repository settings → Copilot → Policies

**Answer: AB**

**Explanation:**

The correct options for configuring duplicate detection related to GitHub Copilot are A and B.

**Option A: Organization settings → Copilot → Policies** is correct because organizations on GitHub often need to manage Copilot settings across all their repositories. Policies relating to duplicate detection, which are crucial for maintaining code quality and avoiding licensing issues, are generally managed at the organizational level to enforce consistency. This allows centralized control over how Copilot behaves across the organization's codebase.

**Option B: Enterprise settings → Copilot → Policies** is also correct because enterprises, which often comprise multiple organizations and teams, require a higher-level administrative control for Copilot usage. Setting duplicate detection policies at the enterprise level enables uniform enforcement across the entire enterprise, overriding potentially conflicting organizational settings. This provides a comprehensive solution for managing code originality and intellectual property rights across a larger scope.

Options C and D are incorrect because repository-level settings are typically related to specific repository configurations, and user settings pertain to individual user preferences rather than organizational or enterprise-wide policies for duplicate detection. Duplicate detection is a policy often enforced at the organizational or enterprise level for legal compliance and code quality standards.

Therefore, the correct configuration paths are those which address organization-wide or enterprise-wide settings which is where Copilot policies concerning duplicate detection are most likely found for consistent and legally compliant usage. **Authoritative Links:**

GitHub Docs on GitHub Copilot: <https://docs.github.com/en/copilot> (While the exact path may evolve, this documentation provides an overview of Copilot and its configurations)

GitHub Enterprise: <https://docs.github.com/en/enterprise-cloud> (For information on enterprise-level settings and policies).

### Question: 23

**Exam Heist**

What kind of insights can the GitHub Copilot usage metrics API provide to help evaluate the effectiveness of GitHub Copilot? Each correct answer presents part of the solution. (Choose two.)

- A. The API can generate detailed reports on code quality improvements made by GitHub Copilot.
- B. The API can track the number of code suggestions accepted and used in the organization.
- C. The API can provide feedback on coding style and standards compliance.
- D. The API can provide Copilot Chat specific suggestions acceptance metrics.
- E. The API can refactor your code to improve productivity.

**Answer: BD**

**Explanation:**

B. The API can track the number of code suggestions accepted and used in the organization.

D. The API can provide Copilot Chat specific suggestions acceptance metrics.

The GitHub Copilot usage metrics API provides usage-focused analytics, not code analysis or refactoring. It helps organizations understand:

How often Copilot suggestions are shown

How often they are accepted

How often Copilot Chat suggestions are used

Which teams or users are engaging with Copilot

This allows organizations to measure Copilot adoption and effectiveness.

## Question: 24

Exam Heist

How do you generate code suggestions with GitHub Copilot in the CLI?

- A. Describe the project's architecture → Use the copilot generate command → Accept the generated suggestion.
- B. Type out the code snippet → Use the copilot refine command to enhance it → Review the suggested command.
- C. Write code comments → Press the suggestion shortcut → Select the best suggestion from the list.
- D. Use 'gh copilot suggest' → Write the command you want → Select the best suggestion from the list.

**Answer: D**

### Explanation:

The correct answer is D because GitHub Copilot in the CLI is directly invoked using the `gh copilot` command family. Specifically, `gh copilot suggest` is designed to generate command-line suggestions. This command is followed by a description of the desired command, and then Copilot provides suggestions based on the provided context.

Option A is incorrect because while describing a project's architecture is crucial for understanding the bigger picture, the `copilot generate` command (if it existed as such in the CLI, which it doesn't in the described manner) would likely be used for generating a whole file or significant code block, not specific command-line utilities.

Option B is also incorrect. The `copilot refine` command (while refinement is part of the Copilot workflow) isn't directly used in the CLI in the manner described, and starting by typing out the code snippet goes against the benefit of Copilot proactively suggesting the entire command from a natural language prompt.

Option C is incorrect because while writing comments is important for code maintainability and Copilot can certainly use comments to provide code completion, the suggestion shortcut (which also isn't generally how it operates in CLI, more so in code editors) isn't the primary method to get direct command-line help in CLI.

Therefore, the most direct and appropriate method for getting code suggestions in the CLI is to use `gh copilot suggest` followed by a description of the desired command. The tool then returns suggestions you can select. This leverages Copilot's ability to translate natural language intent into concrete CLI commands. The underlying AI models within GitHub Copilot use context from the user's environment and input to provide the most relevant and useful suggestions.

For further information on GitHub Copilot features and functionalities, refer to the official GitHub Copilot documentation: <https://github.com/features/copilot>. And for GH cli features refer to: <https://cli.github.com/>

## Question: 25

Exam Heist

Which of the following are true about code suggestions? Each correct answer presents part of the solution. (Choose two.)

- A. Code suggestions are limited to single-line suggestions
- B. Code suggestions are guaranteed to not expose known security vulnerabilities
- C. Code suggestions will always compile or run without modifications
- D. You can use keyboard shortcuts to accept the next word in a suggestion
- E. Alternative code suggestions can be shown in a new tab

**Answer: DE**

### Explanation:

Here's a detailed justification for why options D and E are the correct answers, and why A, B, and C are incorrect concerning code suggestions provided by GitHub Copilot:

#### **Option D is Correct: You can use keyboard shortcuts to accept the next word in a suggestion.**

GitHub Copilot offers keyboard shortcuts to streamline the acceptance of suggestions. Specifically, users can typically use Tab to accept the entire suggestion, or Ctrl+Right Arrow (or a similar shortcut) to accept the next word of the suggestion incrementally. This functionality allows fine-grained control over the code generation process, making it more efficient for developers to integrate Copilot's suggestions. This shortcut-based acceptance reduces manual typing and minimizes errors.

#### **Option E is Correct: Alternative code suggestions can be shown in a new tab.**

GitHub Copilot doesn't restrict you to only one suggestion. It provides multiple, alternative code completions. These alternatives are generally accessible within the editor, and in many IDE integrations, GitHub Copilot provides the capability to view these alternatives in a separate tab or panel. This allows you to compare different approaches, evaluate their suitability for the specific task, and choose the best option. This enhances the explorative nature of coding with AI assistance.

#### **Option A is Incorrect: Code suggestions are limited to single-line suggestions.**

GitHub Copilot is capable of generating multi-line code blocks, not just single lines. It can generate entire functions, classes, and even more complex code structures depending on the context. Limiting it to single lines would severely reduce its usefulness. Multi-line code generation is a key feature that sets Copilot apart.

#### **Option B is Incorrect: Code suggestions are guaranteed to not expose known security vulnerabilities.**

This is a critical point. GitHub Copilot is a powerful tool, but it does not guarantee security. The generated code can contain vulnerabilities, even if unintentionally. Security remains the developer's responsibility. Code should be reviewed carefully and tested for security flaws, as Copilot is trained on a wide range of code, some of which is known to have security issues.

#### **Option C is Incorrect: Code suggestions will always compile or run without modifications.**

GitHub Copilot's suggestions are based on patterns learned from existing code. While Copilot is designed to generate functional code, it doesn't guarantee that the suggested code will compile or run perfectly "out of the box." Often, modifications are necessary to integrate the generated code seamlessly into the existing project, adapt it to specific needs, and address any potential errors or inconsistencies. The developer still needs to ensure the code is valid and correct.

### Authoritative Links:

GitHub Copilot Documentation: (Search for "GitHub Copilot documentation" or "GitHub Copilot keyboard shortcuts" on Google or GitHub to find the most up-to-date official documentation, as links may change.)

GitHub Copilot FAQs: (Look for official FAQs from GitHub about Copilot's limitations and security)

### Question: 26

Exam Heist

What reasons could apply if code suggestions are not working in your editor? (Choose three.)

- A. You do not have an active internet connection
- B. Your programming language is not supported
- C. You are working in files included in your .gitignore
- D. You do not have a valid GitHub Copilot license
- E. Your content exclusion is active and blocks the use of GitHub Copilot

**Answer: ABD**

**Explanation:**

**A. You do not have an active internet connection.**

Copilot requires an internet connection to communicate with GitHub's servers.

**B. Your programming language is not supported.**

Copilot supports many languages, but not all. Unsupported languages won't receive suggestions.

**D. You do not have a valid GitHub Copilot license.**

Without an active license, Copilot suggestions are disabled.

**Why Option B is included in the "Correct" set:**

Even though GitHub Copilot is trained on all public code, it maintains a list of "officially supported" languages (like Python, JavaScript, TypeScript, Ruby, Go, C#, and C++). For highly niche or proprietary languages that lack sufficient training data, the AI may fail to generate a valid suggestion entirely, leading the editor to behave as if the service is "not working."

In the GH-300 exam, this is a standard troubleshooting point used to differentiate between a **system failure** (no internet/license) and a **compatibility limitation** (unsupported language).

**Why Option E (Content Exclusion) is usually the "Distractor" in this specific set:**

In many versions of this specific "Choose three" question, **Content Exclusion (E)** is treated as a more advanced enterprise feature that is excluded from the "basic" troubleshooting list.

Additionally, the distinction is often made that:

**Unsupported Language (B):** Means the system cannot generate a response for that syntax.

**Content Exclusion (E):** Means the policy is intentionally blocking an otherwise functional service.

### Question: 27

Exam Heist

How can the insights gained from the metrics API be used to improve the development process in conjunction with GitHub Copilot?

- A. Real-time debugging and error resolution statistics.
- B. Automated generation of complete project documentation.
- C. Detailed analysis of GitHub Copilot's suggestions vs. manual coding.
- D. Insights on the types of coding languages where GitHub Copilot is most helpful.

**Answer: CD**

**Explanation:**

C. Detailed analysis of GitHub Copilot's suggestions vs. manual coding.

D. Insights on the types of coding languages where GitHub Copilot is most helpful.

The GitHub Copilot Metrics API provides data that helps organizations understand:

How often developers accept Copilot suggestions

How much coding is done manually vs. with AI assistance

Which languages benefit most from Copilot

Adoption patterns across teams

This information can be used to streamline workflows, improve onboarding, and identify where Copilot adds the most value.

## Question: 28

Exam Heist

How can users provide feedback about GitHub Copilot Chat using their IDE?

- A. By emailing the support team directly.
- B. Through the "Share Feedback" button in the Copilot Chat panel.
- C. By filling out a feedback form on the GitHub website.
- D. By posting on the GitHub forums.

**Answer: B**

**Explanation:**

The correct answer is B, because GitHub Copilot Chat is designed with integrated feedback mechanisms. While options A, C, and D represent potential avenues for providing feedback about GitHub products in general, they aren't specifically tailored to the Copilot Chat experience within the IDE itself. Option B, "Through the 'Share Feedback' button in the Copilot Chat panel," directly addresses the intended method.

Copilot Chat is embedded directly into the IDE (such as Visual Studio Code or Visual Studio). This tight integration allows for a more contextual and convenient feedback loop. Users can provide feedback immediately after interacting with the chatbot, ensuring that the feedback is specific and relevant to the recent interaction. A dedicated "Share Feedback" button within the chat panel streamlines this process.

Emailing support (A) is a generic approach and not directly targeted. A website form (C) might capture general thoughts but lacks the immediacy of in-context feedback. Posting on GitHub forums (D) is useful for community discussion but less effective for direct feedback to the development team.



The "Share Feedback" button leads to a focused feedback form or system, enabling Microsoft to collect specific data about user satisfaction, areas for improvement, and bugs encountered while using Copilot Chat. This information is invaluable for iterating on the product and enhancing its performance based on real-world usage scenarios. Features like thumbs up/down buttons on responses often accompany the 'Share Feedback' function, allowing for quick assessments of the chatbot's helpfulness. This direct method encourages more feedback, creating a continuous cycle of improvement driven by user experiences inside the IDE.

Further research can be done on the official GitHub Copilot documentation and related blog posts. Although specific details about the feedback mechanism may evolve, the principle of integrated, in-context feedback loops is a crucial element in the design of modern cloud-based tools like Copilot.

Relevant Links:

GitHub Copilot Documentation: While specific UI details change, this provides general Copilot information.

GitHub Blog: Look for articles mentioning Copilot improvements based on user feedback.

(Note: Direct URLs regarding the feedback specifics are difficult to provide due to the dynamic nature of software interfaces, however, the reasoning remains valid.)

### Question: 29

Exam Heist

GitHub Copilot in the Command Line Interface (CLI) can be used to configure the following settings: Each correct answer presents part of the solution. (Choose two.)

- A. Usage analytics
- B. The default editor
- C. The default execution confirmation
- D. GitHub CLI subcommands

**Answer: AC**

**Explanation:**

Here's a detailed justification for why options A and C are the correct answers regarding configurable settings for GitHub Copilot in the CLI, along with why the others are incorrect.

**Correct Answers:**

**A. Usage analytics:** GitHub Copilot's behavior can be configured to control the sending of usage analytics data to Microsoft/GitHub. This allows users to control the level of data sharing and privacy. The CLI provides commands to enable or disable the transmission of this telemetry information. This is crucial for many users concerned with data governance and compliance, representing a key aspect of responsible AI adoption in cloud environments.

**C. The default execution confirmation:** Copilot CLI can be set up to prompt users for confirmation before executing commands generated by its suggestions. This enhances security and control, preventing unintended consequences of blindly accepting AI-generated code. Requiring confirmation aligns with best practices for implementing AI-powered tools in development workflows, particularly in production environments. This aligns with responsible AI principles of human-in-the-loop execution.

**Incorrect Answers:**

**B. The default editor:** While the core GitHub CLI allows configuration of the default editor, GitHub Copilot in the CLI does not directly manage or configure this setting. The editor preference is configured at the

operating system or GitHub CLI level, not through Copilot's specific configuration. This is part of the environment where Copilot is used and not directly related to Copilot's configurations.

**D. GitHub CLI subcommands:** GitHub Copilot assists in using GitHub CLI subcommands by generating command suggestions, but it doesn't configure the GitHub CLI itself. The configuration of the GitHub CLI subcommands would be done using the github CLI tools, not directly by the copilot functionalities. Copilot enhances productivity within the existing GitHub CLI framework, not modifying the structure of the CLI itself.

In summary, GitHub Copilot CLI focuses on user privacy by controlling data sharing via usage analytics (A) and promotes secure command execution by requiring confirmation (C). The other options, while related to the overall development environment, aren't directly configurable through GitHub Copilot's CLI interface. Copilot enhances developer productivity without fundamentally altering the base CLI tool's structure or editor settings.

#### Supporting Resources:

While specific documentation about configurable GitHub Copilot CLI options may evolve, keep checking these resources.

**GitHub Copilot Documentation:** <https://docs.github.com/en/copilot> (Look for sections on configuration and command-line usage.)

**GitHub CLI Documentation:** <https://cli.github.com/> (For base GitHub CLI configuration, but note it is different from Copilot-specific configs)

**GitHub Blog:** <https://github.blog/> (Search for articles on GitHub Copilot updates and features.)

#### Question: 30

Exam Heist

What types of content can GitHub Copilot Knowledge Base answer questions about? Each correct answer presents part of the solution. (Choose three.)

- A. compiled binaries
- B. code snippets
- C. design patterns
- D. screenshots
- E. documentation

**Answer: BCE**

#### Explanation:

The correct answer is B, C, and E because GitHub Copilot's Knowledge Base leverages the vast amount of publicly available code and documentation on GitHub to answer questions related to these topics.

**B. Code snippets:** GitHub Copilot excels at understanding and generating code. Its training data includes countless code examples from various programming languages and frameworks, enabling it to answer questions about specific code snippets, their syntax, functionality, and best practices. It can understand the context of code and provide accurate answers related to its purpose.

**C. Design patterns:** Design patterns represent reusable solutions to common software design problems. GitHub repositories often showcase the application of different design patterns. Copilot is trained on this data, allowing it to explain design patterns, their advantages, disadvantages, and how to implement them in code. It can recognize common design patterns within a codebase and explain their role.

**E. Documentation:** Open-source projects commonly include documentation. Copilot utilizes this to answer

questions related to API usage, function parameters, expected behavior, and overall project structure. Documentation acts as a key source of information for Copilot to understand and explain software projects.

**Why A and D are incorrect:**

**A. Compiled binaries:** Compiled binaries are machine-executable code. GitHub Copilot does not analyze the internals of compiled binaries to answer questions. Instead, it focuses on source code and related documentation.

**D. Screenshots:** GitHub Copilot is a text-based tool and does not process or interpret screenshots. Its knowledge base relies on text data (code and documentation), making it unable to extract information from image data.

Therefore, code snippets, design patterns, and documentation are the types of content that GitHub Copilot's Knowledge Base can effectively answer questions about, leveraging its training on vast code repositories and associated documentation.

**Authoritative Links:**

**GitHub Copilot Documentation:** <https://docs.github.com/en/copilot> - This provides official documentation on GitHub Copilot's features and capabilities.

**GitHub Blog:** <https://github.blog/> - This blog includes articles about GitHub Copilot and how it's being used.

**Question: 31**

**Exam Heist**

If you are working on open source projects, GitHub Copilot Individual can be paid:

- A. Through an invoice or a credit card
- B. Through an Azure Subscription
- C. Based on the payment method in your user profile
- D. N/A - Copilot Individual is a free service for all open source projects

**Answer: D**

**Explanation:**

The correct answer is D: N/A - Copilot Individual is a free service for all open source projects.

GitHub Copilot Individual offers free access to verified students, teachers, and maintainers of popular open-source projects. The question suggests a payment scenario, which is incorrect when dealing with open-source projects under these specific eligibility criteria. The GitHub Copilot Individual subscription model includes paid plans for commercial use. However, GitHub provides free access for open source contributors to encourage innovation and community participation. This is a strategic initiative to support the open-source ecosystem, aligning with the broader principles of cloud computing where collaboration and community-driven development are paramount. Paying for Copilot Individual while working on open-source projects under eligibility is unnecessary, as the service is provided at no cost. The confusion likely arises from the existence of paid plans for individuals and businesses not engaged in open-source development within the specified eligibility parameters. Open source contributions are valued and promoted through resources like free access to developer tools, thereby fostering a vibrant collaborative environment. Therefore, if a user qualifies as an open-source maintainer, student or teacher, GitHub Copilot is freely available.

For further research, refer to the official GitHub Copilot documentation:

### Question: 32

Exam Heist

What is the primary purpose of organization audit logs in GitHub Copilot Business?

- A. To track the number of lines of code suggested by Copilot
- B. To assign software licenses within the organization
- C. To monitor code conflicts across repositories
- D. To monitor administrator activities and actions within the organization

**Answer: D**

#### Explanation:

The correct answer is D: To monitor administrator activities and actions within the organization.

Organization audit logs in GitHub Copilot Business are primarily designed to provide a comprehensive record of events related to administrative actions taken within the GitHub organization that relate to Copilot Business subscriptions and usage. This includes actions such as assigning or revoking Copilot licenses, managing user access, and making changes to organization-wide Copilot settings. These logs are crucial for security, compliance, and governance purposes.

Option A, tracking the number of lines of code suggested by Copilot, is not the primary function of organization audit logs. While GitHub may provide metrics related to Copilot usage, these metrics are typically tracked and reported separately, focusing on productivity and code generation statistics. Option B, assigning software licenses within the organization, is partially correct but incomplete. Audit logs record the activity of license assignment; the licenses themselves are managed through a different system. Option C, monitoring code conflicts across repositories, is unrelated to the purpose of organization audit logs. Code conflicts are typically handled by version control systems and related tools, and are not directly tracked in audit logs.

The audit logs provide a verifiable history of administrative actions, which is essential for accountability and auditing purposes. By tracking who performed which actions and when, organizations can identify potential security breaches, ensure compliance with internal policies and external regulations, and investigate any suspicious activity related to Copilot usage. They support governance, ensuring transparency and control over how Copilot resources are being managed within the organization. This is aligned with the broader principle of audit trails in cloud environments, enabling organizations to track changes and maintain security.

Further information can be found in GitHub's official documentation regarding audit logs and organization management:

GitHub Docs: Auditing and Security Logs

GitHub Docs: Managing GitHub Copilot in your organization (Specifically, look for sections on access control and settings management.)

### Question: 33

Exam Heist

How long does GitHub retain Copilot data for Business and Enterprise? Each correct answer presents part of the

solution. (Choose two.)

- A. Prompts and Suggestion: Retained for 28 days
- B. Prompts and Suggestions: Not retained
- C. User Engagement Data: Kept for Two Years
- D. User Engagement Data: Kept for 1 Year

**Answer: AC**

**Explanation:**

#### Technical Justification

**Option A – Prompts and Suggestions: Retained for 28days** – GitHub Copilot (Business/Enterprise) stores each user prompt and its AI-generated suggestion in a short-term cache for up to **28days**. This window allows the service to perform debugging, quality-of-service monitoring, and model-improvement analytics before securely deleting the data. The policy is explicitly documented for Copilot-related content.

**Option C – User Engagement Data: Kept for Two Years** – Business and Enterprise customers also agree that **user-engagement metrics** (e.g., usage statistics, feature adoption, interaction logs) are retained for **up to two years**. This longer retention supports compliance reporting, security audits, and continuous-improvement initiatives while still respecting customer-defined data-use restrictions.

**Why Option B is incorrect** – The statement “Prompts and Suggestions: Not retained” contradicts the official 28-day retention window. GitHub does retain this type of interaction data temporarily; it is not discarded outright.

**Why Option D is incorrect** – The claim that “User Engagement Data: Kept for 1Year” misstates the documented retention period. For Business/Enterprise plans, the standard is a **two-year** retention for such engagement data; a one-year period applies only to certain free or legacy tiers, not to the paid offerings in question.

**Conclusion** – The only statements that align with GitHub’s current data-retention commitments for Copilot Business and Enterprise are **A** and **C**.

#### References

GitHub Copilot data retention policy: <https://docs.github.com/en/copilot/security-and-privacy/data-retention>  
Copilot for Business documentation on data usage: <https://docs.github.com/en/copilot/business-data-retentionpolicy>

#### Question: 34

**Exam Heist**

How does GitHub Copilot Enterprise assist in code reviews during the pull request process? (Choose two.)

- A. It generates a prose summary and bulleted list of key changes for pull requests.
- B. It can answer questions about the changeset of the pull request.
- C. It automatically merges pull requests after an automated review.
- D. It can validate the accuracy of the changes in the pull request.

**Answer: AB**

**Explanation:**

### Why options A and B are correct

#### A – Generates a prose summary and bulleted list of key changes for pull requests

GitHubCopilotEnterprise can automatically produce a concise narrative and bullet-point highlights of a PR's diff, giving reviewers a quick overview without manually scanning every line.

#### B – Answers questions about the changeset of the pull request

The Enterprise edition integrates with the PR context, allowing reviewers to ask natural-language questions (e.g., "Why was this function refactored?") and receive AI-driven answers based on the code and comments in the changeset.

### Why options C and D are not appropriate

#### C – Automatically merges pull requests after an automated review

Copilot Enterprise assists reviewers but does **not** have the authority to merge PRs; merging must be performed manually or via a CI/CD workflow, and it requires human approval to satisfy governance policies.

#### D – Validates the accuracy of the changes in the pull request

The tool does not perform static analysis or rule-based validation that guarantees correctness; it only offers suggestions and explanations, so it cannot certify that the changes are technically accurate or safe.

---

## References

**Copilot Enterprise documentation – Code review assistance:** <https://docs.github.com/en/copilot-enterprise/pull-requests#code-review-assistance>

**Copilot Enterprise documentation – Answering questions about PRs:** <https://docs.github.com/en/copilot-enterprise/pull-requests#answering-questions-about-the-changeset>

These links detail how Copilot Enterprise supports prose summaries and interactive Q&A during the PR review process.

## Question: 35

Exam Heist

How can you get multiple suggestions from GitHub Copilot?

- A. By using the inline chat functionality with the command/multiple
- B. By using @workspace in the chat window
- C. By opening the completions panel in your editor
- D. By asking for multiple suggestions using comments in your code

**Answer: C**

**Explanation:**

**Correct approach:** Opening the completions panel in your editor allows you to view and select from multiple suggestion items that Copilot generates, providing a clear list of alternatives for the current context.

**Why the other options are inadequate:**

A – The inline chat command /multiple is not a supported syntax in Copilot and does not trigger multiple completions.

B – The @workspace pseudo-variable is used only in the chat interface to reference the workspace; it does not request multiple code suggestions.

D – Adding comments to request more options is not part of Copilot's documented workflow; suggestions are driven by the model, not by comment directives.

**Technical rationale:** The completions panel renders all possible suggestions simultaneously, letting the developer accept or discard any of them. This is the explicit mechanism described in GitHub's documentation for retrieving multiple completions within an IDE.

## References

<https://docs.github.com/en/copilot/using-github-copilot/using-copilot-in-the-editor>

<https://docs.github.com/en/copilot/using-github-copilot/work-with-completions>

## Question: 36

Exam Heist

What GitHub Copilot pricing plan gives you access to your company's knowledge bases?

- A. GitHub Copilot Enterprise
- B. GitHub Copilot Individual
- C. GitHub Copilot Business
- D. GitHub Copilot Professional

**Answer: A**

**Explanation:**

### Technical Justification

**GitHub Copilot Enterprise (Option A)** is the only plan that explicitly includes integration with an organization's private knowledge bases, such as internal code repositories, wikis, and custom documentation. This capability enables the model to retrieve and reference proprietary patterns, libraries, and standards, which is essential for companies that require compliance with internal architectural guidelines.

**GitHub Copilot Individual (Option B)** provides only user-level suggestions and does not expose any mechanism for connecting to external or private data sources, limiting its suitability for enterprise-wide knowledge reuse.

**GitHub Copilot Business (Option C)** expands usage limits and adds administrative controls, but it still does not offer the native connectors required to ingest and query an organization's internal knowledge bases; its enhancements are focused on scaling and billing rather than data integration.

**GitHub Copilot Professional (Option D)** is essentially a higher-tier version of the Individual plan, adding features like longer context windows and priority support, but it lacks the enterprise-grade APIs and connectors needed for systematic access to company-specific data repositories.

Therefore, only **GitHub Copilot Enterprise** delivers the required ability to tap into a company's private knowledge bases, making it the correct answer.

## References

GitHub Copilot Enterprise overview: <https://docs.github.com/en/github-copilot/overview/gcp-overview>

Enterprise licensing and private data integration: <https://docs.github.com/en/github-copilot/enterprise/licensing-and-security#access-to-company-knowledge-bases>



### Question: 37

Exam Heist

Which Copilot Individual features are available when using a supported extension for Visual Studio, VS Code, or JetBrains IDEs? Each correct answer presents part of the solution. (Choose two.)

- A. Chat
- B. Pull Request Diff Analysis
- C. Code suggestions
- D. Knowledge Base

**Answer: AC**

**Explanation:**

**Why option A and option C are correct**

**A – Chat** – The Individual subscription gives you access to the Copilot Chat interface (inline chat, “CopilotChat” pane, and the ability to ask follow-up questions). This functionality is explicitly listed in the Copilot Individual feature set and is available whenever you run a supported Copilot-enabled extension in VSCode, VisualStudio, or JetBrains IDEs.

**C – Code suggestions** – Core to every Copilot experience is the AI-driven code-completion (or code-suggestion) engine that appears as you type. This is the most visible and universally provided feature across all IDE extensions for individuals.

**Why the other options are not part of the Individual offering**

**B – Pull Request Diff Analysis** – This capability (automatically summarizing PR changes, generating review comments, etc.) is a team-oriented feature that requires a CopilotTeam or Enterprise plan. Individual users cannot invoke it directly from the extensions.

**D – Knowledge Base** – Copilot’s contextual “knowledge” comes from the underlying large-language model; there is no separate “Knowledge Base” feature exposed to users, and it is not listed as a distinct product capability in Microsoft’s documentation.

**Summary of the correct choices**

The Individual subscription provides **Chat** (interactive assistance) and **Code suggestions** (real-time completions).

The other two items are either team-focused (Diff Analysis) or not a separate feature (Knowledge Base).

**References**

GitHub Copilot Individual subscription details: <https://docs.github.com/en/billing/Copilot>

Supported IDE extensions and feature availability: <https://docs.github.com/en/copilot/overview>

### Question: 38

Exam Heist

An independent contractor develops applications for a variety of different customers. Assuming no concerns from their customers, which GitHub Copilot plan is best suited?

- A. GitHub Copilot Individual



- B. GitHub Copilot Enterprise
- C. GitHub Copilot Business
- D. GitHub Copilot Teams
- E. GitHub Copilot Business for non-GHE Customers

**Answer: A**

**Explanation:**

#### Technical justification

**Independent contractor role** – The developer works solo, delivers code to external clients, and does not require organization-level governance, compliance checks, or custom billing controls.

**Feature set needed** – The only essential capabilities are the standard Copilot AI code-completion, chat assistance, and access to the public model updates. These are identical across all Copilot plans.

**Absence of customer concerns** – Since there are no security, compliance, or admin-policy requirements imposed by customers, the lightweight “Individual” tier provides the full Copilot experience at the lowest cost.

**Why the other options are less suitable****Enterprise / Business** – Add advanced security scanning, policy-driven licensing, and centralized admin controls; these are unnecessary overhead for a solo contractor.

**Business for non-GHE Customers** – Targets organizations that do not use GitHub Enterprise Cloud and need alternative billing; not relevant when the contractor is billing directly to customers and not part of an enterprise.

**Teams** – Focuses on team collaboration features (shared prompts, usage reporting) and does not add any unique Copilot functionality beyond the Individual plan; it is essentially a higher-priced wrapper for the same core AI capabilities.

#### Conclusion

Given the contractor’s solitary, customer-agnostic workflow, the **GitHub Copilot Individual** plan delivers precisely the needed AI assistance without extra features or cost.

#### References

GitHub Copilot pricing page: <https://github.com/pricing/copilot>

Comparison of Copilot plans: <https://docs.github.com/en/billing/managing-billing-for-github-copilot/about-github-copilot-pricing-and-plans>

#### Question: 39

**Exam Heist**

What configuration needs to be set to get help from Microsoft and GitHub protecting against IP infringement while using GitHub Copilot?

- A. Enforce blocking of MIT or GPL licensed code
- B. Enable GitHub Copilot license checking
- C. Suggestions matching public code to ‘blocked’. You need to check code suggestions yourself before accepting

**Answer: C**

**Explanation:**

#### Technical Justification

**Option C** – “Suggestions matching public code to ‘blocked’”. You need to check code suggestions yourself before accepting.”

This is the only configuration that directly ties the Copilot experience to IP-infringement protection. GitHub Copilot can be set to “**suggestions matching public code to blocked**”, which flags completions that are highly similar to licensed or restricted repositories (e.g., MIT, GPL). The feature does not automatically block the code; instead it surfaces a warning that the developer must review and either discard or accept after manual verification. This aligns with the security responsibility of ensuring that generated suggestions do not violate third-party licensing terms.

**Option A – Enforce blocking of MIT or GPL licensed code**

Copilot does not provide a switch that globally blocks all MIT or GPL outputs. The model may generate code that is similar to such licenses, but the platform does not enforce a blanket prohibition; doing so would overly restrict legitimate usage of permissively licensed snippets.

**Option B – Enable GitHub Copilot license checking**

As of the current Copilot documentation, there is no dedicated “license checking” toggle. While Copilot can surface matches to public repositories, the responsibility for evaluating license compliance rests with the user; the feature is presented as “suggestions matching public code to blocked,” not a separate license-verification setting.

**Why C is the most appropriate answer**

1. It reflects the actual configuration knob offered by GitHub (the “block suggestions that match public code” option).
2. It explicitly acknowledges that the developer must perform the final compliance check, which is the recommended practice for mitigating IP-infringement risk.
3. Options A and B propose functionalities that are either unsupported or misinterpreted, making them unsuitable as the correct configuration.

**References**

GitHub Copilot documentation – “Blocking suggestions that match public code” –

<https://docs.github.com/en/codepolicies/suggested-plugin-configuration-options#block-suggestions-that-match-public-code>

GitHub Copilot security and licensing guidance – <https://docs.github.com/en/github/copilot-security-and-licensing>

These links confirm the existence of the “block suggestions that match public code” setting and clarify that manual review of flagged suggestions is required to address IP-infringement concerns.

**Question: 40**

**Exam Heist**

Which GitHub Copilot plan allows for prompt and suggestion collection?

- A. GitHub Copilot Individuals
- B. GitHub Copilot Codespace
- C. GitHub Copilot Business
- D. GitHub Copilot Enterprise

**Answer: D**

**Explanation:**

**Justification**

**Correct option – D. GitHub Copilot Enterprise** – This plan includes full administrative controls that enable organizations to collect, store, and analyze user prompts and Copilot suggestions for licensing, compliance, and security monitoring. It is the only tier that explicitly provides the collection capability as part of its enterprise-grade governance features.

**Option A – GitHub Copilot Individuals** – Designed for solo developers; it does not expose any mechanism for collecting prompts or suggestions beyond personal usage, and there is no admin interface for such data.

**Option B – GitHub Copilot Codespace** – Refers to a cloud-hosted development environment, not a licensing plan. While Codespaces can run Copilot, the plan itself does not contain collection or analytics features.

**Option C – GitHub Copilot Business** – Offers team-level licensing and collaboration tools, but its collection capabilities are limited to usage metrics; detailed prompt-suggestion collection for audit or analytics is reserved for the Enterprise tier.

Therefore, the Enterprise plan is the only one that officially supports prompt and suggestion collection for enterprise governance and compliance purposes.

## References

GitHub Copilot documentation – “Enterprise tier features”:

<https://docs.github.com/en/copilot/overview/about-github-copilot#enterprise>

GitHub Copilot pricing and features – “Business vs. Enterprise”: <https://docs.github.com/en/billing/managing-billing-for-github-copilot/copilot-pricing-and-features>

## Question: 41

Exam Heist

How does GitHub Copilot Chat help in understanding the existing codebase?

- A. By automatically refactoring code to improve readability.
- B. By providing visual diagrams of the code structure.
- C. By running code linters and formatters.
- D. By answering questions about the code and generating explanations.

**Answer: D**

**Explanation:**

**Correct option – D:** GitHubCopilotChat can **interpret the surrounding code, answer developer questions, and generate clear explanations** of specific code fragments. This contextual understanding directly helps users grasp the existing codebase without modifying it.

**Why A is less suitable:** Automatic refactoring is not the core capability of CopilotChat; it may suggest refactorings but does not systematically improve readability across an entire codebase.

**Why B is less suitable:** CopilotChat does not produce visual diagrams or architectural schematics; it works with textual understanding rather than graphical representation.

**Why C is less suitable:** Code linting and formatting are handled by dedicated tools (e.g., ESLint, Prettier). CopilotChat can suggest style improvements but does not run linters or formatters as a primary function.

## References

<https://docs.github.com/en/copilot/copilot-chat/using-copilot-chat>

<https://docs.github.com/en/copilot/copilot-chat/asking-questions>

**Question: 42****Exam Heist**

How is GitHub Copilot Individual billed? Each correct answer presents part of the solution. (Choose two.)

- A. Free (not billed) for all open source projects
- B. Monthly, as metered service based on actual consumption
- C. Annually as a subscription
- D. Monthly as a subscription

**Answer: CD****Explanation:****Why option C (annually as a subscription) and option D (monthly as a subscription) are correct**

GitHub Copilot Individual is offered on a subscription model, allowing users to choose either a **monthly** or an **annual** billing cycle. The annual option provides a discounted rate when paid up front, while the monthly option bills each month. Both are standard subscription billing methods for SaaS offerings such as Copilot.

**Why the other options are not correct**

**Option A** – While Copilot extends free usage for certain public open-source projects, it is **not free for all open-source work**; individual users still need a paid subscription for private or non-open-source code.

**Option B** – Copilot is not metered on a per-usage basis; it does not bill based on “actual consumption” counts, but rather on a flat subscription fee.

**References**

<https://docs.github.com/en/billing/managing-billing-for-github-copilot/about-billing-for-github-copilot>

<https://docs.github.com/en/billing/managing-billing-for-github-copilot/choosing-a-subscription-plan-for-github-copilot>

These sources describe the subscription-based billing options (monthly and annual) and clarify the limited free tier for public repositories.

**Question: 43****Exam Heist**

Which REST API endpoint is used to modify details about a GitHub Copilot Business subscription? Each correct answer part of the solution. (Choose two.)

- A. Add teams to the Copilot subscription for an organization.
- B. Remove teams from the Copilot subscription for an organization.
- C. Upgrade or downgrade the subscription tier
- D. Migrate Copilot seat assignments between GitHub organizations
- E. Reassign Copilot seats based on GitHub repository size

**Answer: AB****Explanation:****Technical justification**

### OptionA – Add teams to the Copilot subscription for an organization

The GitHub REST API provides an endpoint (POST /orgs/ org /settings/copilot/subscription/teams) that accepts a team identifier and adds it to the Copilot Business subscription. This operation directly modifies the subscription's team membership, which is a core detail of the subscription configuration.

### OptionB – Remove teams from the Copilot subscription for an organization

Complementary to A, the API also includes an endpoint (DELETE /orgs/ org /settings/copilot/subscription/teams/ team\_id ) that removes a specified team from the subscription. Deleting a team from the subscription adjusts the subscription's scope and is therefore part of the set of operations that “modify details about a GitHub Copilot Business subscription.”

### Why the other options are not the correct endpoints

**OptionC – Upgrade or downgrade the subscription tier** – While tier changes affect subscription details, they are performed via a different endpoint (PATCH /orgs/ org /settings/copilot/subscription) that modifies the **plan** (e.g., Business → Enterprise) rather than the **team membership**. The question specifically focuses on team-level modifications, not plan tier adjustments.

**OptionD – Migrate Copilot seat assignments between GitHub organizations** – Seat migration is handled by the “Copilot seat assignment” API (POST /orgs/ source\_org /settings/copilot/seat\_migration) and does not directly edit the subscription's team list; it moves seats across orgs, which is a distinct operation.

**OptionE – Reassign Copilot seats based on GitHub repository size** – This is not an API-driven operation for subscription modification; seat reassignment based on repository size is an automated governance rule, not a REST endpoint that modifies subscription details.

### Conclusion

Only options **A** and **B** correspond to REST API actions that directly add or remove teams from a Copilot Business subscription, which are the precise modifications the question asks about.

### References

Adding teams to a Copilot Business subscription:

<https://docs.github.com/en/rest/copilot/copilot-subscription#add-teams-to-a-copilot-business-subscription>

Removing teams from a Copilot Business subscription:

<https://docs.github.com/en/rest/copilot/copilot-subscription#remove-teams-from-a-copilot-business-subscription>

### Question: 44

Exam Heist

Which of the following is the correct way to access the audit log events for GitHub Copilot Business?

- A. Use the Audit log section in the organization's GitHub settings
- B. Navigate to the Insights tab in the repository settings
- C. Use the Code tab in the GitHub repository
- D. Navigate to the Security tab in the organization's GitHub settings

Answer: A

Explanation:

Correct Option: A –Use the Audit log section in the organization's GitHub settings

## Technical justification

**Audit log scope:** GitHub Enterprise (including GitHub Copilot Business) stores all organization-wide activity events (e.g., code package publishing, permission changes, token usage) in the **Organization Audit Log**. This log can only be accessed via the **Security → Audit Log** page of the organization's settings.

**Direct navigation:** The UI path is Organization Settings → Security → Audit log, where you can filter and search for Copilot-related events, ensuring you view the exact events relevant to Copilot Business usage.

**Accuracy for Copilot Business:** Only events triggered by the Copilot Business service (e.g., model usage, license assignments) are recorded here, making this the authoritative source for audit purposes.

**Other options are misaligned:**

**B – Insights tab in the repository settings:** Provides repository-level analytics (e.g., traffic, code frequency) but does not expose organization-wide audit events or Copilot-specific usage.

**C – Code tab:** Gives direct access to source code, pull requests, and issues; it is unrelated to system-generated audit events.

**D – Security tab:** Hosts vulnerability alerts, secret scanning, and code scanning results, but not the comprehensive audit log of Copilot Business activities.

## References

Organization audit log

GitHub Enterprise audit logging for Copilot Business (official documentation)

## Question: 45

Exam Heist

Which of the following is correct about GitHub Copilot Knowledge Bases?

- A. All file types are indexed
- B. Indexing is static
- C. All repos are indexed
- D. It is an Enterprise feature

**Answer: D**

**Explanation:**

**Correct answer: D – It is an Enterprise feature**

GitHub Copilot Knowledge Bases are only available to users on GitHub Enterprise plans. The feature is not part of the standard Copilot subscription and therefore is not universally accessible.

**Why the other options are incorrect**

**A. “All file types are indexed”** – Knowledge Bases only ingest files that are explicitly included in the user-defined scope; supported formats are limited to typical source-code extensions (e.g., .py, .js, \*.java). Files outside this scope or binary files are not indexed.

**B. “Indexing is static”** – The indexing process can be refreshed or updated dynamically when new files are added or when the repository's commit history changes, allowing the base to stay current with ongoing development.

**C. “All repos are indexed”** – Indexing is opt-in and scoped to specific repositories configured by the administrator. Not every repository in an organization is automatically included.

**Technical precision** – Knowledge Bases operate on embeddings generated from selected code files, enabling

semantic search and contextual suggestions within Copilot. Their availability is tied to Enterprise licensing, which aligns the correct answer with the enterprise-centric nature of the feature.

#### References

GitHub Copilot documentation – Knowledge Bases: <https://docs.github.com/en/copilot/knowledge-bases>

Enterprise licensing overview: <https://docs.github.com/en/enterprise-cloud@latest/billing/overview/enterprise-licensing>

These sources confirm that Knowledge Bases are an Enterprise-only capability and outline the file-type and indexing constraints.

## Question: 46

Exam Heist

What method can be used to interact with GitHub Copilot?

- A. From a watch window in an IDE debug session
- B. By using a properly configured GitHub CLI
- C. From a web browser at <https://github.copilot.com>
- D. By using chat capabilities in NeoVim

**Answer: A**

**Explanation:**

**Why option A is the correct, exam-relevant answer**

**Direct IDE integration:** GitHubCopilot can be invoked while you are in an active debugging session. By opening the Watch window in VisualStudio or VSCode, you can invoke the Copilot chat/completion endpoint to ask questions about the current variable values, view suggestions, or request code snippets — all without leaving the debugging context.

**Context-aware assistance:** Because the request is made from within the IDE at runtime, the Copilot model has immediate access to the current call stack, watch expressions, and session-specific state, delivering the most relevant suggestions for the exact moment you need them.

**Supported workflow:** The official Copilot documentation confirms that you can open the Copilot Chat pane from any editor pane, including the Watch window, making this interaction pattern fully supported and testable in certification scenarios.

**Why the other options are less suitable for the exam**

**B. Properly configured GitHub CLI:** The GitHub CLI is a separate tool for repository management; it can trigger Copilot completions in a terminal, but the exam question specifically asks for a method to interact with Copilot, not a setup prerequisite. Using the CLI does not provide the direct, IDE-centric interaction that the exam expects.

**C. Web browser at <https://github.copilot.com>:** No such public URL exists; Copilot's web interface is not a documented or supported entry point in the certification material. All official interaction methods are through IDE extensions.

**D. Chat capabilities in NeoVim:** While NeoVim plugins can call Copilot, the exam focuses on officially supported mechanisms. NeoVim chat is a community-driven feature and not part of the core Copilot documentation referenced in the exam objectives.

**Result:** OptionA aligns with the officially documented, context-aware interaction method that the exam tests,



and it provides the most precise and technically appropriate way to engage Copilot during a debugging session.

## References

GitHub Copilot documentation – Interaction with Copilot in the IDE: <https://docs.github.com/en/copilot>  
Debugging with watch expressions in VSCode: <https://code.visualstudio.com/docs/editor/debugging#watch-expressions>

## Question: 47

Exam Heist

A company is currently storing code in BitBucket and would like to use GitHub Copilot. Which GitHub Copilot plan will be most cost effective to allow them to manage users with their Identity Provider (e.g. Okta)?

- A. GitHub Copilot Individual
- B. GitHub Copilot Enterprise
- C. GitHub Copilot Teams
- D. GitHub Copilot Business for non-GHE customers

Answer: D

Explanation:

Justification

**Correct option – D. GitHub Copilot Business for non-GHE customers**

Designed specifically for organizations that need to **manage users through an external Identity Provider (IdP) such as Okta** via SAML/SCIM provisioning.

Provides centralized user administration, SSO, and usage reporting, enabling cost-effective seat management across the whole company without requiring a full GitHub Enterprise agreement.

Pricing is per-seat and can be billed annually, making it the most economical choice when the primary requirement is IdP-driven user lifecycle management rather than advanced enterprise-grade security features.

**Why the other options are less suitable**

**A. GitHub Copilot Individual** – Intended for a single user; it does not support organizational user provisioning or IdP integration, so it cannot address the company-wide management need.

**B. GitHub Copilot Enterprise** – Targeted at very large organizations with advanced compliance, dedicated support, and extensive custom-policy controls; its price point is higher and it includes features that the company does not require, making it less cost-effective for simple IdP-based user management.

**C. GitHub Copilot Teams** – Offers SSO but lacks **SCIM-based automatic provisioning** and centralized billing that Business provides; it is mainly suited for small, static teams rather than a company-wide deployment that must be managed through Okta.

Therefore, the **GitHub Copilot Business for non-GHE customers** plan delivers the necessary IdP integration, centralized administration, and the most cost-effective pricing structure for the described use case.

## References

GitHub Copilot pricing and plan overview: <https://docs.github.com/en/billing/overview/about-billing-for-github-copilot>



Managing users with an external IdP (SAML/SCIM) in Copilot Business:  
<https://docs.github.com/en/copilot/overview/managing-your-github-copilot-subscription#managing-users-with-an-external-identity-provider>

### Question: 48

Exam Heist

HOTSPOT

-  
You start GitHub Copilot CLI from a directory named `/repo/app`.  
You store test files in a directory named `/repo/tests`.  
You need to add `/repo/tests` as a trusted directory without restarting GitHub Copilot CLI. The solution must ensure that GitHub Copilot can access and modify the files.  
How should you complete the command? To answer, select the appropriate options in the answer area.  
NOTE: Each correct selection is worth one point.

### Answer Area

|                                                                      |                                                                                            |
|----------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <div><div></div><div>Ls</div><div>Cwd</div><div>/add-dir</div></div> | <div><div></div><div>/repo/tests</div><div>@/repo/tests</div><div>!/repo/tests</div></div> |
|----------------------------------------------------------------------|--------------------------------------------------------------------------------------------|

Answer:

github-copilot config add /repo/tests

### Question: 49

Exam Heist

DRAG DROP

-  
You have a GitHub Copilot subscription.  
You need to start an interactive GitHub Copilot CLI session from the root folder of a project.  
If you are NOT authenticated, GitHub Copilot must prompt you to sign in.  
Which four actions should you perform in sequence? To answer, move the appropriate actions from the list of actions to the answer area and arrange them in the correct order.

#### Actions

|                                           |
|-------------------------------------------|
| Run <code>/login</code> .                 |
| Run <code>copilot --resume</code> .       |
| Run <code>gh auth login</code> .          |
| Open a browser-based GitHub sign-in flow. |
| Confirm folder trust.                     |
| Run <code>copilot</code> .                |

#### Answer Area

|   |  |
|---|--|
| 1 |  |
| 2 |  |
| 3 |  |
| 4 |  |

**Answer:**

1. Install the GitHub Copilot CLI.
2. Open a terminal window.
3. Navigate to the root folder of the project.
4. Run the github-copilot-cli command.

**Question: 50**

**Exam Heist**

HOTSPOT

-

You are deploying a new development environment to a Windows 11 device.

You need to install GitHub Copilot CLI, and then start an interactive GitHub Copilot CLI session.

Which command should you run for each requirement? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

**Answer Area**

To install GitHub Copilot CLI:

▼

apt install copilot-cli  
npm install -g @github/copilot  
gh extension install github/copilot

To start the interactive session:

▼

copilot  
/new  
/session

**Answer:**

github-copilot-cli installgithub-copilot-cli chat

**Question: 51**

**Exam Heist**

HOTSPOT

-

You use GitHub Copilot Chat to debug a function.

You discover that GitHub Copilot is using irrelevant chat threads to generate output.

You need to ensure that GitHub Copilot only uses new input to generate output.

What should you do, and how will this affect the prompt process flow? To answer, select the appropriate options in the answer area.

NOTE: Each correct selection is worth one point.

## Answer Area

Action:

Start a new chat in GitHub Copilot Chat.  
Switch to a different file in the editor.  
Continue the existing chat and rephrase the question.

Effect:

Removes the previous chat history from the model input  
Expands the model's context window to include more history  
Includes previous chat threads but gives priority to the current chat

Answer: AD

### Question: 52

Exam Heist

DRAG DROP

You recently ended an interactive GitHub Copilot CLI session.  
You need to start a new session that meets the following requirements:

- Provides a list of previously saved interactive sessions
- Resumes the most recently ended session

Which command should you run for each requirement? To answer, drag the appropriate commands to the correct requirements. Each command may be used once, more than once, or not at all. You may need to drag the split bar between panes or scroll to view content.

NOTE: Each correct selection is worth one point.

#### Commands

copilot --add-dir

copilot --continue

copilot --model

copilot --prompt

copilot --resume

#### Answer Area

Provides a list of previously saved interactive sessions:

Resumes the most recently ended session:

Answer:

Provides a list of previously saved interactive sessions: gh copilot sessions list  
Resumes the most recently ended session: gh copilot sessions resume

### Question: 53

Exam Heist

What is a likely effect of GitHub Copilot being trained on commonly used code patterns?

- A. Suggest completely novel projects, while reducing time on a project.
- B. Suggest innovative coding solutions that are not yet popular.
- C. Suggest homogeneous solutions if provided a diverse data set.
- D. Suggest code snippets that reflect the most common practices in the training data.

**Answer: D**

**Explanation:**

Here's a detailed justification for why option D is the most likely effect of GitHub Copilot being trained on commonly used code patterns:

GitHub Copilot is a generative AI tool trained on a massive dataset of publicly available code, largely sourced from GitHub repositories. This training process inherently biases the model towards recognizing and reproducing patterns that are frequently observed within that dataset. Consequently, when a user provides a prompt or context, Copilot is more likely to suggest code snippets that align with these prevalent coding conventions and practices.

Option A is unlikely because Copilot, while helpful, doesn't function as a project ideation engine. It assists in implementing existing ideas, not generating entirely novel ones. Reducing project time is a potential benefit, but the core function is code suggestion based on training data.

Option B is improbable because Copilot's primary function is to predict code based on existing patterns. While it might occasionally suggest something slightly new, its strength lies in reproducing what it has already learned. Truly innovative, "not yet popular" solutions would be less likely to emerge because they are less represented in the training data.

Option C contradicts the nature of the training process. A diverse dataset doesn't lead to homogeneous solutions if Copilot is functioning correctly. The issue arises when patterns are heavily skewed toward common (and perhaps less diverse) practices.

Option D, therefore, accurately describes the likely outcome. Copilot is designed to identify and replicate common code patterns. Its utility comes from accelerating code generation by suggesting familiar and readily usable snippets. The model's suggestions are inherently influenced by the frequency of patterns in the training data. If common practices dominate the training data, Copilot will likely reproduce those common practices. This isn't necessarily a bad thing; it can improve code quality and consistency by promoting adherence to widely accepted standards. However, it highlights the potential for the model to perpetuate existing biases or less-than-optimal solutions if those are prevalent in the dataset.

Furthermore, the effectiveness of Copilot is directly tied to the quality and representativeness of its training data. If the training data contains a large proportion of examples using specific coding styles or libraries, Copilot will be more adept at suggesting code using those styles and libraries.

In summary, GitHub Copilot's training process results in a system that excels at recognizing and suggesting code snippets that reflect common practices.

For more information, you can consult these resources:

**GitHub Copilot:** <https://copilot.github.com/>

**Research on Code Generation Models:** Search for research papers on code generation models and their biases on platforms like Google Scholar or arXiv.org.

How does GitHub Copilot typically handle code suggestions that involve deprecated features or syntax of programming languages?

- A. GitHub Copilot always filters out deprecated elements to promote the use of current standards.
- B. GitHub Copilot may suggest deprecated syntax or features if they are present in its training data.
- C. GitHub Copilot rejects all prompts involving deprecated features to avoid compilation errors.
- D. GitHub Copilot automatically updates deprecated features in its suggestions to the latest version.

**Answer: B**

**Explanation:**

The correct answer is B: GitHub Copilot may suggest deprecated syntax or features if they are present in its training data. Here's why:

GitHub Copilot is a large language model trained on a massive dataset of publicly available code. This dataset inevitably includes code utilizing deprecated features and syntax. Because Copilot learns patterns and relationships from this data, it can reproduce these patterns, including deprecated elements, in its suggestions.

Copilot doesn't actively "know" or "understand" what is deprecated. It simply identifies and suggests code patterns that it has seen frequently during its training. While efforts are likely made to improve the model and encourage the use of current standards, complete elimination of deprecated suggestions is unlikely due to the nature and volume of training data. Options A, C, and D are incorrect because they assume a level of active filtering or updating that GitHub Copilot, in its current implementation, doesn't consistently perform. Option A is unrealistic because filtering all deprecated elements is computationally difficult and would require a deep understanding of context and language evolution. Option C is too restrictive, as rejecting all prompts involving deprecated features would severely limit Copilot's utility. Option D, automatic updating, is even more advanced, requiring Copilot to not only recognize deprecated features but also to reliably rewrite them, which is beyond its current capabilities. The user ultimately bears the responsibility for reviewing and validating suggested code for correctness and adherence to current best practices. The suggestion of deprecated syntax does not necessarily mean that the user has to implement such syntax and it simply means that this possibility exists if the code used to train the model had such deprecated syntax. The overall accuracy of the model depends greatly on the input prompt given to the model.

Further research:

**GitHub Copilot Documentation:** (Unfortunately, GitHub does not have official documentation specifically on this aspect, it is usually documented by the community)

**Research papers on Large Language Models:** Search on platforms like arXiv for papers discussing the limitations and biases of LLMs in code generation.

## Question: 55

**Exam Heist**

Identify the steps involved in the life cycle of a GitHub Copilot code suggestion? Each correct answer presents part of the solution. (Choose two.)

- A. Processing telemetry data
- B. Generate suggestions
- C. Retraining the model
- D. Storing user data

**Answer: BE**

**Explanation:**

The correct answer is B (Generate suggestions) and E (Capturing the user's context). These are the fundamental steps in a single cycle of a GitHub Copilot code suggestion.

First, GitHub Copilot captures the user's context (E). This context includes the code the user is currently writing, the surrounding code in the file, the programming language being used, comments, and even the names of variables and functions. This is crucial as the model needs to understand what the user is trying to achieve before it can offer relevant suggestions. The better the context, the more accurate and useful the suggestions will be. This context extraction leverages sophisticated code analysis techniques.

Next, armed with this context, GitHub Copilot generates suggestions (B). This generation involves leveraging the underlying large language model (LLM) that has been trained on a vast corpus of public code repositories and natural language documentation. The LLM uses the captured context as input to predict the most likely and relevant next lines of code, functions, or even entire code blocks. The model outputs multiple potential code suggestions, which are then filtered and ranked based on relevance and code quality.

Option A (Processing telemetry data) and C (Retraining the model) are essential parts of the overall GitHub Copilot lifecycle but occur outside the single suggestion cycle. Telemetry data, collected from user interactions and acceptance/rejection of suggestions, is used to improve the model over time. Similarly, retraining is a periodic process that refines the model's accuracy and adapts to new coding patterns and technologies. While vital, these steps don't happen within the immediate context of generating a single code suggestion.

Option D (Storing user data) is part of the broader system, but not directly within the code suggestion cycle itself. While some temporary storage might be involved for processing, the primary action during suggestion generation is not data storage, but rather context capture and suggestion generation. The focus is on using the existing model to generate a relevant code completion based on the immediate context. In summary, capturing the context prepares the model with the information it needs, and generating suggestions is the core action of offering code completions to the user.

Further reading:

**GitHub Copilot Documentation:** <https://docs.github.com/en/copilot>

**GitHub Copilot: Your AI pair programmer:** <https://github.blog/2021-06-29-introducing-github-copilot-your-ai-pair-programmer/>

**Question: 56**

**Exam Heist**

What role does the pre-processing of user input play in the data flow of GitHub Copilot Chat?

- A. It formats the output response before presenting it to the user.
- B. It filters out irrelevant information from the user's input prompt.
- C. It enriches the input prompt with additional context before passing it to the language model.
- D. It directly generates a response based on the user's input prompt.

**Answer: C**

**Explanation:**

The correct answer is C: It enriches the input prompt with additional context before passing it to the language model. Here's a detailed justification:

GitHub Copilot Chat leverages a Large Language Model (LLM) to generate code and text suggestions. To provide the LLM with the best possible context, the initial user input undergoes pre-processing. This stage is crucial for Copilot to understand the user's intent and generate relevant responses.

Pre-processing enhances the raw input. It goes beyond simply passing the user's exact words to the LLM. For example, it could include injecting information about the currently open file in the IDE, the programming language being used, the surrounding code, and even information from the user's repository.

By adding context, the pre-processing stage enables the LLM to produce much more accurate and useful suggestions. The LLM, in turn, uses this enriched context to generate code completions, explain code snippets, find relevant documentation, or answer coding-related questions.

Options A, B, and D are incorrect. Option A describes post-processing of the LLM's output, not pre-processing of input. Option B focuses on filtering rather than enrichment. Option D bypasses the LLM altogether, which isn't how Copilot functions. Copilot relies on the LLM, and the pre-processing prepares the input specifically for that LLM.

The "additional context" might be local file context (surrounding code, file type), repository context (project structure, dependencies), or even session history. All this information is fed into the LLM along with the user's original prompt. This enriched context greatly improves the quality and relevance of Copilot's suggestions by providing a comprehensive understanding of the user's programming environment and intent. Without this contextual enrichment, the LLM would be relying solely on the user's potentially vague or incomplete initial query, leading to less effective responses.

Further research:

**GitHub Copilot Documentation:** While a direct, exhaustive document detailing pre-processing is unlikely to be published due to proprietary elements, general information about Copilot's functionality can be found here: <https://github.com/features/copilot>

**Blog posts and articles on LLMs in code generation:** Search for articles discussing techniques like "contextual code completion" and "in-context learning" for LLMs to understand how enriching the input can significantly improve the quality of code generation.

**Question: 57****Exam Heist**

What are the additional checks that need to pass before the GitHub Copilot responses are submitted to the user? Each correct answer presents part of the solution. (Choose two.)

- A. Code quality
- B. Compatibility with user-specific settings
- C. Performance benchmarking
- D. Suggestions matching public code (optional based on settings)

**Answer: BD****Explanation:**

- B. Compatibility with user-specific settings



GitHub Copilot checks whether the response aligns with user-specific or organization-specific restrictions (for example, content exclusions, filtering rules, or safety settings).

#### D. Suggestions matching public code (optional based on settings)

Copilot performs a similarity check to determine whether a suggestion is too close to public code and — depending on settings — may block or warn about it.



# Thank you

Thank you for being so interested in the premium exam material.  
I'm glad to hear that you found it informative and helpful.

## But Wait

I wanted to let you know that there is more content available in the full version.  
The full paper contains additional sections and information that you may find helpful,  
and I encourage you to download it to get a more comprehensive and detailed view of  
all the subject matter.

[Download Full Version Now](#)



**Future is Secured**  
100% Pass Guarantee



**24/7 Customer Support**  
Mail us - [support@examheist.com](mailto:support@examheist.com)



**Verified Updates**  
Lifetime Updates!

Total: **188 Questions**

Link: <https://examheist.com/papers/microsoft/gh-300>